



MBARARA UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF COMPUTING AND INFORMATICS

GROUP 8

**DESIGN AND IMPLEMENTATION OF A RELATIONAL UNIVERSITY LIBRARY
MANAGEMENT SYSTEM.**

Student Name: Komujuni Emelyn

Registration Number: 2025/BCS/105/PS

Course Name: Database Systems.

Course code: CSC1204

Course Year: One

Academic Year: 2025/2026

1. SUMMARY.

This report describes the design and build of a modern University Library Management System. Right now, the library has many problems because they use a manual paper-based system. It is hard for the staff to keep track of books, members, and late returns.

To solve this, I have designed a relational database using MySQL. This system manages six main areas: Members, Librarians, Books, Categories, Loans, and Fines. The main goal of this system is to make the library run faster and keep the data safe. I have added a security feature where Librarians must use a password to access the system. This stops unauthorized people from changing library records. This report covers the business reasons for the system, the logic used to build the tables, and the final implementation code.

2. BUSINESS CASE

2.1 The Application Domain

The application domain for this project is a University Library environment. The system supports essential daily tasks such as inventory cataloging, member registration, loan processing, and fine tracking. It serves two primary user groups:

- **Librarians:** Administrative users who need reliable tools to check books in/out, register new students, and monitor overdue items.
- **Members** (Students/Faculty): Users who borrow books and need efficient service without long waiting times.

2.2 Analysis of Benefits

Transitioning to a relational database provides significant advantages over the manual system:

- **Tangible Benefits:**
 - **Reduced Resource Loss:** By strictly linking every book to a specific loan record, the university reduces the financial cost of lost or unreturned items.

- **Operational Efficiency:** Automated fine calculation reduces the man-hours required for administrative tasks, allowing librarians to focus on assisting students.
- **Intangible Benefits:**
 - **Data Integrity:** Replacing paper records eliminates human transcription errors and prevents files from being "lost".
 - **Accountability:** Every transaction is tied to a specific Librarian's ID, establishing a clear audit trail.

2.3 Analysis of Costs

To ensure the project is economically feasible, the following costs were considered:

- **Hardware:** Procurement of a central database server and terminal workstations at the checkout desk.
- **Software:** Utilizing MySQL provides a robust, open-source backend, keeping software licensing costs low.
- **Intangible Costs (Time):** The time required for system design, SQL coding, testing, and debugging.

2.4 User Issues and Adoption

The shift to a digital system requires an adoption effort. Librarians accustomed to paper logs will face a "learning curve." To mitigate this, the user interface will be designed with clear, simple dialogue structures, and a brief training manual will be provided to ensure staff feel confident using the new software.

2.5 Risk Assessment

- **Technical Risks:** Database downtime or corruption. *Mitigation:* Implementing daily SQL backups to secure servers.
- **Security Risks:** Unauthorized access to member data. *Mitigation:* Requiring mandatory Librarian passwords and utilizing database-level Foreign Key constraints to prevent accidental data deletion.

2.6 System Limitations

A candid assessment reveals that the current system manages physical inventory only. It does not yet support digital E-book downloads or online remote reservations for students.

3. LOGICAL AND PHYSICAL DESIGN

3.1 Logical Design: List of Entities

The logical structure of the database comprises six core entities:

1. **Book:** The physical assets of the library (Title, Author).
2. **Member:** The individual borrowing the book.
3. **Librarian:** The staff member authorizing the transaction.
4. **Category:** A classification entity to group books by subject.
5. **Loan:** An associative, transactional entity linking a Member, a Book, and a Librarian.
6. **Fine:** A financial entity triggered by overdue loans.

3.2 Attributes and Multiplicities

To define the business rules, the following attributes and multiplicities (relationships) were established:

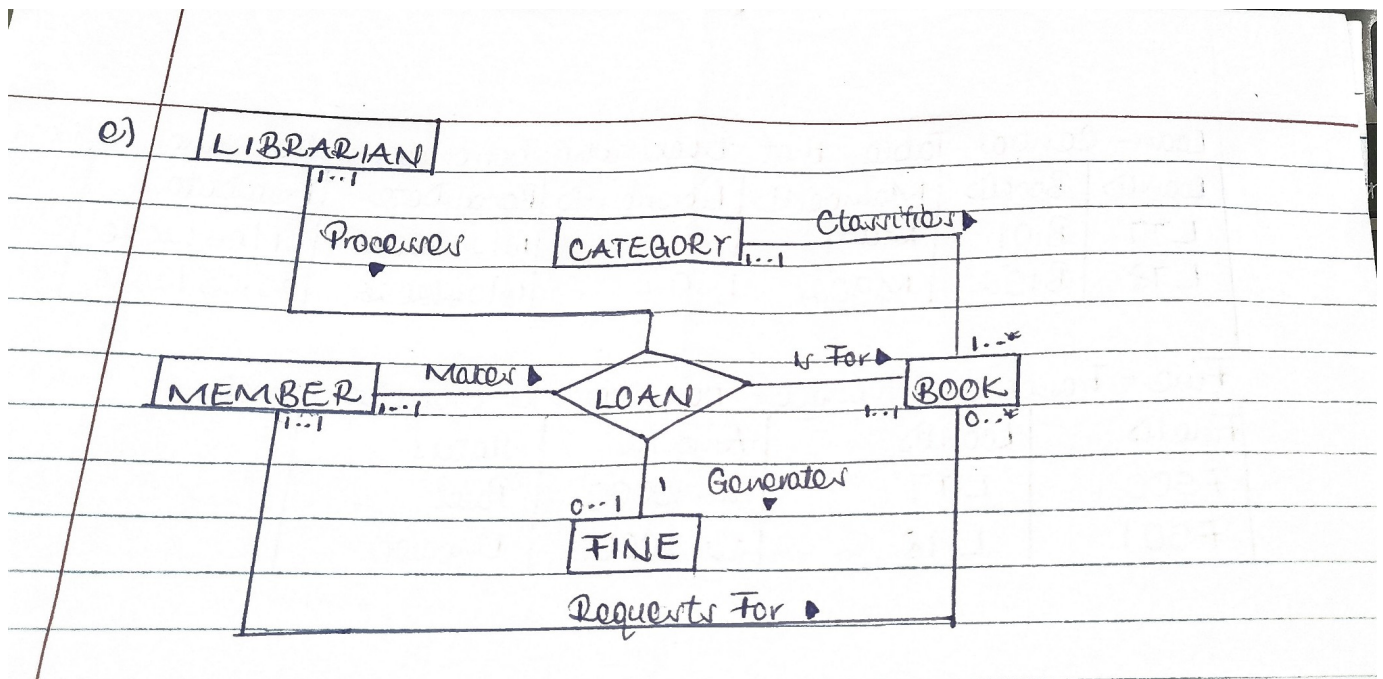
- Category to Book (1:N): One category (e.g., "Computer Science") contains many books, but each book has one primary category.
- Member to Loan (1:N): One member can have multiple loan records over time.
- Book to Loan (1:N): While a book is checked out to one person at a time, the database stores the history of *all* its past loans.
- Librarian to Loan (1:N): One librarian processes many loans during a shift.
- Loan to Fine (1:1): One specific loan record can generate a maximum of one fine if returned late.
- Member to Book

Data Dictionary:

- **Book:** BookID (PK), Title, Author, CategoryID (FK).
- **Member:** MemberID (PK), FullName, Email.
- **Librarian:** LibrarianID (PK), StaffName, Shift, Password.
- **Category:** CategoryID (PK), CategoryName.
- **Loan:** LoanID (PK), BookID (FK), MemberID (FK), LibrarianID (FK), BorrowDate, DueDate.
- **Fine:** FineID (PK), LoanID (FK), Amount, Status.

3.3 Comprehensive Entity Relationship Diagram (ERD)

(Note: The logical architecture described above is visually represented in the comprehensive ER Diagram below, establishing the relational links and cardinalities of the system.)



3.4 User Interface (Dialogue Structure)

The system UI is structured as a simple, secure dialogue flow for the Librarian:

1. Login Screen: Librarian enters LibrarianID and Password.

2. Main Dashboard: Presents a menu with options: *Manage Members*, *Process Loan*, *View Overdue Books*.
3. Loan Processing Screen: Librarian inputs MemberID and BookID. The system automatically stamps the BorrowDate to minimize data entry errors.

3.5 Physical Design: Database Schema

The physical layout in MySQL translates the logical entities into interconnected tables using Foreign Keys. The Category and Librarian tables act as independent lookup tables, while the Loan table serves as the central junction linking the relational architecture together.

```
mysql> USE university_project;
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_university_project |
+-----+
| book                          |
| category                     |
| fine                         |
| librarian                    |
| loan                         |
| member                       |
+-----+
6 rows in set (0.00 sec)

mysql> DESCRIBE loan;
+-----+
| Field      | Type | Null | Key | Default | Extra |
+-----+
| LoanID     | int  | NO   | PRI | NULL    | auto_increment |
| BookID     | int  | YES  | MUL | NULL    |
| MemID      | int  | YES  | MUL | NULL    |
| LibID      | int  | YES  | MUL | NULL    |
| BorrowDate | date | YES  |     | NULL    |
| DueDate    | date | YES  |     | NULL    |
+-----+
6 rows in set (0.04 sec)

mysql> SELECT * FROM member;
+-----+
| MemID | FullName | Email |
+-----+
| 1     | John Aine | john@univ.ac.ug |
| 2     | Jane Babirye | jane@univ.ac.ug |
| 3     | Musa Ali | musa@univ.ac.ug |
+-----+
```

3.6 Real-World System Comparison

As part of the design process, I researched the Koha Integrated Library System, an open-source LMS used globally.

- Insight Gained: I learned how Koha strictly separates structural data (Categories) from transactional data (Loans). I applied this by ensuring my system uses an independent Category table, which prevents spelling errors and data anomalies when cataloging new books. Furthermore, tracking fine statuses (Paid/Unpaid) was inspired by Koha's financial modules.

4. IMPLEMENTATION

4.1 Creating the Database Objects

The physical design was implemented in MySQL using the following script.

SQL

```
CREATE DATABASE UniversityLibrary;
```

```
USE UniversityLibrary;
```

```
CREATE TABLE Category (  
    CategoryID INT PRIMARY KEY AUTO_INCREMENT,  
    CategoryName VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Librarian (  
    LibrarianID INT PRIMARY KEY AUTO_INCREMENT,  
    StaffName VARCHAR(100) NOT NULL,  
    Shift VARCHAR(20),  
    Password VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE Member (  

```

```

MemberID INT PRIMARY KEY AUTO_INCREMENT,
FullName VARCHAR(100) NOT NULL,
Email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Book (
    BookID INT PRIMARY KEY AUTO_INCREMENT,
    Title VARCHAR(100) NOT NULL,
    Author VARCHAR(100),
    CategoryID INT,
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);

CREATE TABLE Loan (
    LoanID INT PRIMARY KEY AUTO_INCREMENT,
    BookID INT,
    MemberID INT,
    LibrarianID INT,
    BorrowDate DATE NOT NULL,
    DueDate DATE NOT NULL,
    FOREIGN KEY (BookID) REFERENCES Book(BookID),
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
    FOREIGN KEY (LibrarianID) REFERENCES Librarian(LibrarianID)
);

CREATE TABLE Fine (
    FineID INT PRIMARY KEY AUTO_INCREMENT,
    LoanID INT UNIQUE,
    Amount DECIMAL(10,2) NOT NULL,
    Status VARCHAR(20) DEFAULT 'Unpaid',
    FOREIGN KEY (LoanID) REFERENCES Loan(LoanID)
);

```


);

4.2 Data Population

Sample data was loaded to test the system relationships and ensure foreign key constraints function properly.

SQL

```
INSERT INTO Category (CategoryName) VALUES ('Computer Science'), ('History');
INSERT INTO Librarian (StaffName, Shift, Password) VALUES ('Admin', 'Morning', 'Pass123');
INSERT INTO Member (FullName, Email) VALUES ('Emelyn Komujuni', 'emelyn@university.ac.ug');
INSERT INTO Book (Title, Author, CategoryID) VALUES ('Database Systems', 'C.J. Date', 1);
```

-- Processing a sample loan

```
INSERT INTO Loan (BookID, MemberID, LibrarianID, BorrowDate, DueDate)
VALUES (1, 1, 1, '2026-05-01', '2026-05-15');
```

4.3 Sample Queries and Results

To demonstrate system capabilities, the following key outputs were generated:

Query 1: Find all books in a specific category (e.g., Computer Science).

SQL

```
SELECT Book.Title, Book.Author
FROM Book
JOIN Category ON Book.CategoryID = Category.CategoryID
WHERE Category.CategoryName = 'Computer Science';
```

Query 2: Identify all active loans and the responsible librarian.

SQL

```
SELECT Member.FullName, Book.Title, Loan.DueDate, Librarian.StaffName
FROM Loan
```

JOIN Member ON Loan.MemberID = Member.MemberID

JOIN Book ON Loan.BookID = Book.BookID

JOIN Librarian ON Loan.LibrarianID = Librarian.LibrarianID;

4.4 Implementation Conclusion

The implementation phase successfully transitioned the theoretical design into a functional relational database. By enforcing referential integrity through foreign keys, the system prevents common data entry errors, such as assigning a loan to a non-existent member. The use of the `AUTO_INCREMENT` attribute ensures that every record is uniquely identifiable without manual indexing, making the system scalable for future university growth.

The successful execution of the sample queries proves that the system is capable of providing real-time administrative insights, such as tracking overdue books and identifying librarian shifts, which solves the core problems of the previous manual system.

5. BIBLIOGRAPHY

1. **Date, C. J. (2004).** *An Introduction to Database Systems* (8th ed.). Pearson/Addison-Wesley.
(This text provided the foundational logic for the relational integrity and normalization used in this project).
2. **Elmasri, R., & Navathe, S. B. (2016).** *Fundamentals of Database Systems* (7th ed.). Pearson.
(Used for referencing Entity-Relationship modeling and cardinalities).
3. **MySQL AB. (2026).** *MySQL 8.0 Reference Manual*. Available at: <https://dev.mysql.com/doc/>.
(Consulted for SQL syntax, specifically for JOIN operations and Foreign Key constraints).
4. **Koha Community.** *Koha Open Source Library System Documentation*. Available at: <https://koha-community.org/>. (Used as a real-world comparison for the "Category" and "Fine" module logic).